TP 2 : portfolio interactif React (Vite)

L'application fournie

Un portfolio interactif développé avec **React** (Vite) : une page à sections (à propos, projets, contact).

- Section **Projets** filtrable par catégorie (Tous, Web, Mobile, Design)
- Section **Contact** avec un formulaire (nom, email, message) qui valide l'adresse email côté client et affiche un message d'erreur ou de confirmation
- Suite de tests **Vitest** + **React Testing Library** déjà écrite (`src/components/ContactForm.test.jsx`)
- Aucun backend ni base de données : l'application est entièrement statique une fois compilée (`npm run build` produit `dist/`)
- Config qualité de code déjà en place : **ESLint**, configuré dans `eslint.config.js`

Consultez `projet-react/README.md` et `projet-react/COMMANDES.md` pour tout faire tourner en local avant de toucher au moindre fichier de workflow. Vous devez comprendre ce que fait l'application avant d'automatiser son test et son déploiement.

Votre mission

Construire, sur un dépôt Git contenant ce projet, **deux pipelines GitHub Actions** :

1. `ci.yml` , qui teste le code, vérifie sa qualité, et produit un build de production à chaque changement
2. `deploy.yml` , qui déploie automatiquement sur votre conteneur de groupe, seulement si la CI est passée

Partie 1 : CI (`.github/workflows/ci.yml`)

1. Déclenchement

Le pipeline doit se déclencher sur `push` vers `main` , et sur `pull_request` vers une branche `develop` que vous créerez.

2. Cache des dépendances npm

Node installe ses dépendances avec `npm` , à partir d'un fichier `package-lock.json` . Recherchez quel dossier npm utilise pour stocker son cache, et mettez-le en cache avec une clé basée sur `package-lock.json` .

3. Installation de l'environnement Node.js

Utilisez l'action officielle permettant d'installer une version de Node.js sur le runner (cherchez son nom exact sur le GitHub Marketplace), puis installez les dépendances du projet avec la commande recommandée en environnement CI.

4. Exécution des tests et build : deux jobs, deux artefacts

Organisez le pipeline en deux jobs distincts.

Le job `test` installe les dépendances, exécute la suite de tests Vitest, et archive le résultat en artefact (nommez-le clairement, par exemple `rapport-tests`), même en cas d'échec.

Le job `build` doit dépendre du premier (déclarez-le avec la dépendance appropriée) et ne se lancer que si les tests ont réussi. Il réinstalle les dépendances (chaque job démarre sur une machine neuve), exécute la commande de build de production, puis archive le dossier `dist/` généré dans un **second artefact, distinct du premier**, avec un nom clair (par exemple `build-production`).

Point de vigilance : ne confondez pas les deux artefacts. Le premier contient un rapport de tests (utile pour vérifier ce qui a été validé), le second contient le résultat compilé de l'application (utile pour un déploiement ultérieur). Les mélanger dans un seul artefact rendrait leur exploitation confuse.

5. Qualité de code

Ajoutez, dans le job `test`, une étape qui exécute **ESLint** sur le projet. Le job doit échouer si cette commande échoue.

6. Résumé du pipeline

Dans le job `build`, ajoutez une étape qui écrit, dans le résumé du job, un titre Markdown récapitulant les étapes du pipeline (tests, qualité, build) et mentionnant les deux artefacts produits.

7. Badge de statut

Ajoutez, dans le `README.md` du projet, le badge de statut du workflow.

Partie 2 : CD (`.github/workflows/deploy.yml`)

Une fois la CI en place et verte, ajoutez le déploiement automatique vers votre conteneur de groupe.

1. Déclenchement conditionné à la CI

Le déploiement ne doit se lancer qu'après la réussite du workflow de CI ci-dessus (`workflow_run`), et seulement sur la branche `main`.

2. Connexion SSH au conteneur de groupe

Connectez-vous en SSH à votre serveur de groupe pour y exécuter les commandes de déploiement. Les informations de connexion (hôte, utilisateur, port, clé privée) ne doivent jamais apparaître en clair dans le workflow : stockez-les sous forme de secrets, sous les noms de votre choix.

3. Clone ou mise à jour du dépôt sur le serveur

Premier déploiement : le dossier `~/app` n'existe pas encore côté serveur, il faut cloner. Déploiements suivants : `~/app` existe déjà, il suffit de faire un `git pull`. Écrivez la condition qui distingue les deux cas (indice : un dépôt Git a toujours un dossier `.git`), sinon le tout premier déploiement échoue.

4. Démarrage de l'application

Une fois le code présent sur le serveur, démarrez l'application avec `docker compose up -d --build`.

Contraintes

- Les deux pipelines doivent être fonctionnels et visibles avec un statut vert dans l'onglet Actions de votre dépôt
- Le badge doit refléter le statut réel du dernier run de CI